

1. Intelligent agents

AI = agents that get *percepts* (sensors) and do *actions* (actuators). Agent = program + architecture; the **agent function** maps percept history  $\mapsto$  action, and given PEAS it fully specifies the agent. **Rational**: maximizes *expected* performance measure given percepts so far (not actions in). **PEAS**: **P** what is scored (safety, speed, legal, comfort); **E** world it acts in; **A** outputs (steering, brake, text); **S** inputs (camera, lidar, touch screen, chat history). Chess app: **P** win/draw, human-like ELO; **E** board, rules, user; **A** move/resign; **S** touch.

| Environment properties: definition $\rightarrow$ exam cue                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                        |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Full / partial</b>                                                                                                                                                                                                                                                                                                                                         | Sensors give the complete state <i>relevant to the performance measure</i> , every step. <i>Full</i> : “reads the full document”, board games. <i>Partial</i> : camera/lidar blind spots, hidden cards, no feedback on failures.                                                       |
| <b>Det / stoch</b>                                                                                                                                                                                                                                                                                                                                            | Next state fully set by current state + action. <i>Det</i> : “guaranteed to produce the desired outcome”. <i>Stoch</i> : may fail, random people/obstacles, disruptions, dice, “25% chance”. Partial obs. can <i>look</i> non-det.                                                     |
| <b>Episodic / seq</b>                                                                                                                                                                                                                                                                                                                                         | Episodic: atomic perceive $\rightarrow$ act episodes; next one does not depend on earlier actions (“no bearing on the next”, “self-contained tasks”, classify each item). Sequential: current choice affects future ones (tracks deliveries, booked room stays taken, chess, driving). |
| <b>Static / dynamic</b>                                                                                                                                                                                                                                                                                                                                       | Static: unchanged while the agent deliberates. Dynamic: people move, disruptions “at any time”. <i>Semi-dynamic</i> : world static but score changes with time (timed chess).                                                                                                          |
| <b>Discrete / cont</b>                                                                                                                                                                                                                                                                                                                                        | Finite distinct <i>state, time, percepts, actions</i> (judge each). Real-space navigation, steering angle, speed $\rightarrow$ continuous. Folders, board squares, timesteps $\rightarrow$ discrete.                                                                                   |
| <b>Single / multi</b>                                                                                                                                                                                                                                                                                                                                         | X counts as an agent if its behaviour is best described as maximizing a measure that depends on other agents’ behaviour. Tic-tac-toe, driving: multi.                                                                                                                                  |
| <b>Known / unknown</b>                                                                                                                                                                                                                                                                                                                                        | Known: outcomes of all actions (rules) given. Independent of observability: Solitaire = known + partial; new video game buttons = unknown + full.                                                                                                                                      |
| <b>Worked</b> : tic-tac-toe = FO, det, static, discrete, seq, multi. Hospital delivery robot (24/25 S2) = <b>stoch</b> , <b>cont</b> (also PO, seq, dyn). Doc sorter (25/26 S1) = <b>FO</b> , <b>episodic</b> , <b>static</b> (det, discrete). Meeting-room scheduler (25/26 S2) = <b>stoch</b> , <b>seq</b> , <b>dyn</b> ; PO and cont. accepted either way. |                                                                                                                                                                                                                                                                                        |

| Classification traps                                                                                                                                                                             |  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| FO $\nrightarrow$ episodic (chess). Stochastic $\neq$ partially obs. Judge from the agent’s view and stated measure; quote the deciding phrase. Search assumes FO, det, static, discrete, known. |  |

**Agent structures**: simple **reflex** (condition–action rules on current percept only)  $\subset$  **model-based reflex** (keeps internal state)  $\subset$  **goal-based** (searches for actions reaching a goal)  $\subset$  **utility-based** (utility ranks states, handles trade-offs). Mixes allowed (AlphaGo = utility + learning). **Learning agent**: *performance element* picks actions from current knowledge; *critic* scores vs a performance standard  $\rightarrow$  feedback; *learning element* improves the performance element; *problem generator* proposes exploratory actions. **Exploration vs exploitation** (unknown env.): learn more about the world vs maximize gain with current knowledge (new dish vs usual Indomie).

2. Search formulation

Specify state space; initial  $s_0$ ; actions  $A(s)$ ; transition  $T(s, a)$ ; goal test; step cost  $c(s, a, s')$  and path cost  $g(n)$ .

**State space** = all configurations; **search space** = the part actually explored. **Problem graph** = states/actions; **search tree** = paths tried. **State representation** must be complete: every real configuration has a representation (Hanoi: 3 tuples,  $3^n$  states). **State** = world configuration. **Node** = state + parent/action/path/depth/cost. Different nodes may hold same state. **Solution** = action path to goal; **optimal** = minimum path cost. Problem-solving loop: formulate goal  $\rightarrow$  formulate problem  $\rightarrow$  search  $\rightarrow$  execute. **Complete**: finds a solution if one exists. **Optimal**: returned solution has minimum cost.

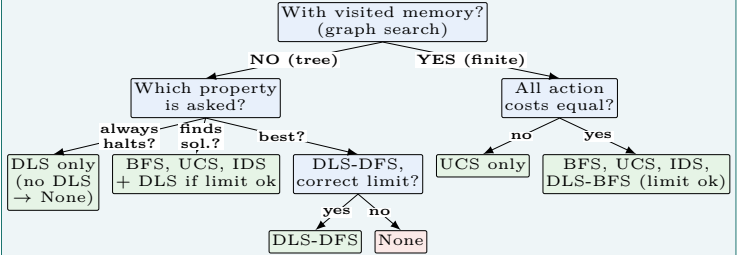
Generic search / frontier

**Tree search** can repeat states. **Graph search** tracks visited/best  $g$ ; update/reopen if cheaper path appears unless guarantees make it unnecessary. Let  $b$  = max branching;  $d$  = shallowest goal depth;  $m$  = max depth;  $\ell$  = depth limit;  $C^*$  = optimal cost.

| Uninformed search: NO visited vs WITH visited                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                              |                                  |             |                     |                      |          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|----------------------------------|-------------|---------------------|----------------------|----------|
| Without visited (tree search) = may re-expand a state $\Rightarrow$ reversible actions make cycles, so the tree is infinite. With visited (graph search) = never re-expand a state $\Rightarrow$ finite state space: every algorithm halts and detects “no solution”; but the visited set costs $O(\#states)$ space, which cancels the space advantage of DFS and IDS. Infinite space (all integers): visited alone does not guarantee halting.                                                                                                                                                                                                                                                                                              |                              |                                  |             |                     |                      |          |
| Alg.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | NO visited                   |                                  | visited*    | optimal             | time                 | space    |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | finds sol?                   | halts if none?                   | finds+halts |                     |                      |          |
| BFS                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | yes                          | no                               | yes         | unit cost           | $b^d$                | $b^d$    |
| UCS                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | yes ( $c > 0$ )              | no                               | yes         | any $c > 0$         | $b^{1+C^*/\epsilon}$ | same     |
| DFS                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | no (loops)                   | no                               | yes         | $\text{no}^\dagger$ | $b^m$                | $bm$     |
| DLS-DFS                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | iff $\ell \geq \text{depth}$ | <b>YES</b>                       | yes         | $\text{no}^\dagger$ | $b^\ell$             | $b\ell$  |
| DLS-BFS                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | iff $\ell \geq \text{depth}$ | <b>YES</b>                       | yes         | unit cost           | $b^\ell$             | $b^\ell$ |
| IDS                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | yes                          | no ( $\ell \rightarrow \infty$ ) | yes         | unit cost           | $b^d$                | $bd$     |
| *finite state space. $^\dagger$ optimal anyway if every solution has the same cost (e.g. unique goal, all at depth $M^2 - 2N$ ). Unit cost: BFS = UCS (UCS adds a log for the PQ). IDS/BFS return the <i>shallowest</i> , not the <i>cheapest</i> . <b>Space col.</b> = <b>frontier only</b> ; with <b>visited</b> , all $O(\#states)$ . Exact: BFS $O(b^{d+1})$ if goal-test on pop. <b>Rank</b> : $d \leq m$ ( $m$ may be $\infty$ ); $d \leq C^*/\epsilon < 1 + C^*/\epsilon$ (unit cost: $C^*/\epsilon = d$ ); $m$ vs $C^*/\epsilon$ no fixed order; $\ell < d \Rightarrow$ DLS misses. <b>Time</b> : BFS = IDS $b^d \leq$ UCS $b^{1+C^*/\epsilon}$ ; DFS $b^m \geq b^d$ . <b>Space</b> : IDS $bd \leq$ DFS $bm \ll$ BFS $b^d \leq$ UCS. |                              |                                  |             |                     |                      |          |

Search-analysis MCQs: question  $\rightarrow$  answer

**Reachable by  $> 1$  action sequence?**  $\rightarrow$  **Yes** if any action can be undone or actions can be reordered.  
**Many goal states?**  $\rightarrow$  Yes if the goal ignores something (final floor) or several configs pass. No if the goal is one exact value/config.  
**Unique optimal path?**  $\rightarrow$  No if actions can be reordered.  
**Always a solution?**  $\rightarrow$  **No** if some parameter can block it (capacity 0, unreachable config). Yes if you can always step  $\pm 1$  to the goal.  
**Best algorithm? decision tree** (best = complete, optimal, efficient, detects no solution):



**Tick all in a leaf. Why: DLS only**: its cap is the only thing that stops cycles. **BFS/UCS/IDS**: level by level, cycles can’t trap them. **DLS-DFS**: only one that always halts; finds if limit ok. **None**: limit too small  $\rightarrow$  nothing halts and finds. **UCS**: only optimal one with unequal costs. **BFS/UCS/IDS/DLS-BFS**: the visited set counts as space (TA, 30/9), so all are exponential space; IDS loses its space edge but ties on big-O time and stays optimal (sl. 68).  
**Search tree finite?** (no visited, solution exists)  $\rightarrow$  **all but DFS**: BFS, UCS, IDS, DLS stop; DFS loops.  
**Is the DLS limit right?** Plug in extremes (huge capacity with 1 passenger; negative numbers). Limit  $<$  the worst solution depth  $\rightarrow$  wrong  $\rightarrow$  DLS is out.  
**Read EVERY action cost** ( $\times 2$  costs 2): unequal  $\Rightarrow$  **UCS only** (IDS/BFS: shallowest  $\neq$  cheapest).

3. A\* and heuristics

$f(n) = g(n) + h(n)$ ;  $h^*(n)$  = true optimal remaining cost. **Admissible**:  $0 \leq h(n) \leq h^*(n)$  for every  $n$  (never overestimates). Enough for optimal/complete  $A^*$  without visited memory under standard conditions.

**Consistent**: for each successor  $n'$ ,  $h(n) \leq c(n, a, n') + h(n')$ ,  $h(\text{goal}) = 0$ . Consistency  $\Rightarrow$  admissibility; supports  $A^*$  with/without visited memory. It makes  $f$  nondecreasing along a path.

**Dominance**: admissible  $h_1 \geq h_2$  everywhere  $\Rightarrow h_1$  dominates/more informed.  $h = 0 \Rightarrow$  UCS.

| Heuristic admissibility by move set |                                            |                      |             |
|-------------------------------------|--------------------------------------------|----------------------|-------------|
| moves (unit cost)                   | admissible, tight                          | over-estimates?      |             |
| 4-dir grid                          | Manhattan $ \Delta x  +  \Delta y $        | –                    |             |
| 8-dir (diag cost 1)                 | Chebyshev $\max( \Delta x ,  \Delta y )$   | Manhattan, Euclidean | <b>do</b>   |
| any direction                       | Euclidean                                  | –                    |             |
| 6-dir 3D                            | $ \Delta x  +  \Delta y  +  \Delta z $     | –                    |             |
| 26-dir 3D                           | $\max( \Delta x ,  \Delta y ,  \Delta z )$ | 3D Manhattan         | <b>does</b> |

**Order: Chebyshev  $\leq$  Euclidean  $\subset$  Manhattan**. Admissible iff  $h \leq$  the obstacle-free optimal cost for *that* move set. So for 4-dir all three are admissible (Manhattan is tightest); for 8-dir only Chebyshev is. Cost  $c$  per move: multiply by  $c$  (the *cheapest* move cost if costs vary).  $\max(h_1, h_2)$  of admissible heuristics is admissible and dominates both;  $h_1 + h_2$  is not, in general. Build one: relax a rule (remove walls, allow teleport or diagonal moves) and take that problem’s optimal cost.

Search trace recipe (course pseudocode)

Frontier table columns: **node** | **path** |  $g$  |  $h$  |  $f$ , one row per pop.  
1. **Pop** by rule: BFS FIFO, DFS LIFO, UCS min  $g$ , Greedy min  $h$ ,  $A^*$  min  $f = g + h$  (apply the stated tie rule).  
2. **Goal test on pop**, not on generation (the course code tests right after pop(), for every algorithm).  
3. With visited memory: if the popped state is already visited, **skip it**; else mark it visited.  
4. Push *all* successors (duplicates allowed in the frontier) with parent  $g$  + edge cost.  
**Count** “nodes expanded” = pops that were not skipped and not the goal (check the question’s definition).  
**Consistency check**: for every directed edge  $n \rightarrow n'$ :  $h(n) - h(n') \leq c(n, n')$ , and  $h(\text{goal}) = 0$ .  
**Admissibility check**:  $h(n) \leq h^*(n)$ ; compute  $h^*$  backwards from the goal (a Dijkstra-style table).

4. Local search

Moves among complete candidate solutions; little memory; returns a configuration, not a path. Generally incomplete, not globally optimal. Formulate: state, initial state, neighbours  $N(s)$ , evaluation  $E(s)$  (max/min), stopping rule.

**Steepest-ascent hill climbing**: choose best neighbour; move only if strictly better; stop at local optimum/plateau. Failure modes: local maximum, plateau, ridge. Remedies: random restart, sideways moves, stochastic choice, larger/random neighbourhood. **Simulated annealing**: pick a random neighbour; always take it if better, else accept with prob.  $e^{\Delta E/T}$ , which shrinks as the move gets worse and as temperature  $T$  cools.

**Local beam**: keep best  $k$  of all successors of  $k$  states (shared info, unlike  $k$  restarts); diversity may collapse.

**Reasonable local search?** ( $\infty$  restarts) **Tick all that**: hold the candidate (path, or node+visited)  $\cdot$  adjacent + unvisited moves  $\cdot$  eval = real objective. **Reject**: no visited set, non-adjacent jumps, wrong eval. MRT  $\Rightarrow$  **C, D**.

TSP formulation

State = city permutation; neighbour = 2-opt segment reversal; cyclic cost  $C(s) = \sum_{i=1}^{n-1} d(c_i, c_{i+1}) + d(c_n, c_1)$ ; maximize  $E = -C$ . Do not omit return edge.

5. Adversarial search

Finite deterministic two-player zero-sum perfect-information game: states, player-to-move, actions/transition, terminal test, terminal utility  $U$  from MAX’s view.  $V(s) = U(s)$  if terminal,  $\max_{s'} V(s')$  at MAX,  $\min_{s'} V(s')$  at MIN. Minimax assumes optimal play. Root action is  $\arg \max$  of successor values. Time  $O(b^m)$ ; depth-first memory polynomial. Never maximize at MIN nodes.

Alpha-beta pruning

**Visit a child**  $\rightarrow$  pass your **current**  $\alpha, \beta$  to it. **Child returns  $v$  to MAX**  $\rightarrow \alpha = \max(\alpha, v)$ . **to MIN**  $\rightarrow \beta = \min(\beta, v)$ .  $\alpha \geq \beta \rightarrow$  skip this node’s remaining children.

6. Supervised ML foundations

Dataset  $D = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$ ,  $\mathbf{x}^{(i)} \in \mathbb{R}^d$ . **Regression**: real-valued target. **Classification**: categorical target.  
Keep separate: **data**; **model**  $\mathbf{h}_w$ ; **loss**  $J(\mathbf{w})$  optimized in training; **performance metric** used to judge; **learning algorithm** choosing  $\mathbf{w}$ . Loss need not equal evaluation metric.  
**Train** fits parameters; **validation** chooses hyperparameters/models; **test** gives one final unbiased estimate. Repeated test inspection = leakage. Assumption for generalization: train/test i.i.d. from same distribution.

Metrics

Regression:  $\text{MSE} = \frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$ ,  $\text{MAE} = \frac{1}{n} \sum_i |\hat{y}_i - y_i|$ . MSE penalizes large errors more; MAE is less outlier-sensitive.  
Classification (positive class must be specified): Accuracy =  $\frac{TP+TN}{N}$ , Precision  $P = \frac{TP}{TP+FP}$ , Recall  $R = \frac{TP}{TP+FN}$ ,  $F_1 = \frac{2PR}{P+R}$ . Precision: among predicted +, fraction correct. Recall: among actual +, fraction found. Accuracy may conceal minority-class failure.  
**Confusion matrix**  

|          |                     |                     |
|----------|---------------------|---------------------|
|          | pred +              | pred -              |
| actual + | True Positive (TP)  | False Negative (FN) |
| actual - | False Positive (FP) | True Negative (TN)  |

  
Row = truth, column = prediction. FP = false alarm (type I); FN = miss (type II).  
 $P = \frac{TP}{\text{pred}+}$  (column),  $R = \frac{TP}{\text{actual}+}$  (row).  
*Ex:* TP 3, FN 1, FP 2, TN 4: Acc = 0.7,  $P = \frac{3}{5} = 0.6$ ,  $R = \frac{3}{4} = 0.75$ ,  $F_1 = \frac{2(.6)(.75)}{1.35} = 0.667$ .  
Always-predict-negative on 99% negative data: Acc 0.99,  $R = 0$ .

Fit and complexity

**Underfit**: too simple/high bias; training and test error high. **Overfit**: learns noise/high variance; very low train error but worse unseen error. More complexity usually lowers training error but can raise test error. More suitable data can reduce variance; cannot fix wrong features/severe mismatch automatically.  
**Parameters** learned from training (weights, tree splits/leaf predictions). **Hyperparameters** configured/tuned on validation (depth, min leaf, learning rate, polynomial degree, batch size, iterations).

7. Decision trees

Internal node tests attribute; branch = outcome; leaf predicts class. Prediction follows one root-to-leaf path. Learning recursively chooses splits on subsets; greedy, interpretable, can overfit.  
 $H(Y) = -\sum_k p_k \log_2 p_k$  ( $0 \log 0 := 0$ ). Binary:  $H = 0$  if pure;  $H = 1$  at  $p = \frac{1}{2}$ .  
 $H(Y | X) = \sum_x P(X = x) H(Y | X = x)$ ;  $IG(Y; X) = H(Y) - H(Y | X)$ . Choose available attribute with largest  $IG$ . Conditional entropy is a *size-weighted* average of children.  $IG = 0$ : observed attribute gives no class information; pure children maximize gain.

Recursive learner

- If examples empty: return parent/default majority.
- If all labels same: return that class leaf.
- If no attributes: return current majority.
- Else choose max- $IG$  attribute, split by values, remove categorical attribute on that path, recurse.

Greedy choice need not yield globally smallest/best tree. Decision trees can represent any Boolean function, but finite/noisy data may not reveal it.

**DT build recipe (exam format)**  
**Root**: 1.  $H(\text{labels})$  via the lookup table. 2. For each attribute: count labels per value. Remainder =  $\sum \frac{|\text{branch}|}{n} H(\text{branch})$ . 3.  $IG = H - \text{rem}$  to 2 d.p. 4. Max  $IG$ ; if tied, use the **given priority order**.  
**Each child**, check in this order:  
(a) 0 rows  $\rightarrow$  *parent's* majority (the given default only at the initial call).  
(b) all one label  $\rightarrow$  that label.  
(c) no attributes left  $\rightarrow$  majority; tied  $\rightarrow$  “Undecided” (if stated).  
(d) else recompute  $IG$  on *this subset only*, for the remaining attributes.  
Recurse on **every value in the domain**, including values with 0 rows.  
**Speed**: a pure branch adds 0 to the remainder; an attribute that makes every branch pure has  $IG = H$ .  $-p \log_2 p = p \log_2 \frac{1}{p}$ .

Pruning / unseen values

**Leaf  $\geq k$** : any child  $k \rightarrow$  cut the **whole split** (repeat up).  
**Pre-pruning**: stop early (max depth / min samples). More pruning: more bias, less variance. Unseen value  $\rightarrow$  stated default (usually majority).

**Entropy arithmetic**  
Remainder =  $\sum \frac{|\text{child}|}{n} H(\text{child})$ ;  $IG = H(\text{parent}) - \text{remainder}$ . Weight by child size, never a plain average.

**Entropy lookup by raw counts (a yes / b no)**  
Row = #yes, column = #no (symmetric, so either way round). e.g. 3Y, 5N  $\rightarrow$  row 3, col 5 = .954. Any count 0  $\rightarrow H = 0$ .  

|     |      |      |      |      |      |      |      |      |      |      |      |      |
|-----|------|------|------|------|------|------|------|------|------|------|------|------|
| a\b | 1    | 2    | 3    | 4    | 5    | 6    | 7    | 8    | 9    | 10   | 11   | 12   |
| 1   |      | .918 | .811 | .722 | .650 | .592 | .544 | .503 | .469 | .439 | .414 | .391 |
| 2   | .918 |      | .971 | .918 | .863 | .811 | .764 | .722 | .684 | .650 | .619 | .592 |
| 3   | .811 | .971 |      | .985 | .954 | .918 | .881 | .845 | .811 | .779 | .750 | .722 |
| 4   | .722 | .918 | .985 |      | .991 | .971 | .946 | .918 | .890 | .863 | .837 | .811 |
| 5   | .650 | .863 | .954 | .991 |      | .994 | .980 | .961 | .940 | .918 | .896 | .874 |
| 6   | .592 | .811 | .918 | .971 | .994 |      | .996 | .985 | .971 | .954 | .937 | .918 |
| 7   | .544 | .764 | .881 | .946 | .980 | .996 |      | .997 | .989 | .977 | .964 | .949 |
| 8   | .503 | .722 | .845 | .918 | .961 | .985 | .997 |      | .998 | .991 | .982 | .971 |
| 9   | .469 | .684 | .811 | .890 | .940 | .971 | .989 | .998 |      | .998 | .993 | .985 |
| 10  | .439 | .650 | .779 | .863 | .918 | .954 | .977 | .991 | .998 |      | .998 | .994 |
| 11  | .414 | .619 | .750 | .837 | .896 | .937 | .964 | .982 | .993 | .998 |      | .999 |
| 12  | .391 | .592 | .722 | .811 | .874 | .918 | .949 | .971 | .985 | .994 | .999 |      |

**Exact, one line on the calculator** ( $n = a + b$ ; drop terms with count 0):  
 $H(a, b) = \frac{n \ln n - a \ln a - b \ln b}{n \ln 2}$  e.g.  $\frac{9 \ln 9 - 4 \ln 4 - 5 \ln 5}{9 \ln 2} = 0.99108$   
 $IG = \frac{N(\text{parent}) - \sum N(\text{child})}{n_{\text{parent}} \ln 2}$   
*Ex:* (4, 7)  $\rightarrow$  (0, 2), (4, 5):  $\frac{(11 \ln 11 - 4 \ln 4 - 7 \ln 7) - (9 \ln 9 - 4 \ln 4 - 5 \ln 5)}{11 \ln 2} = 0.13477 \rightarrow$  **0.13** (table: 0.14).  
Use when a table answer is borderline at 2 d.p. (third decimal 4 or 5).

8. Linear regression

Add dummy  $x_0 = 1$ :  $\mathbf{h}_w(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} = \sum_{j=0}^d w_j x_j$ . MSE ( $\frac{1}{2}$  simplifies derivative):  
 $J(\mathbf{w}) = \frac{1}{2n} \sum_i \left( \mathbf{h}_w(\mathbf{x}^{(i)}) - y^{(i)} \right)^2$ ,  $\frac{\partial J}{\partial w_j} = \frac{1}{n} \sum_i \left( \mathbf{h}_w(\mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}$ . Vector form ( $X \in \mathbb{R}^{n \times (d+1)}$ ):  $\nabla J = \frac{1}{n} X^\top (X\mathbf{w} - \mathbf{y})$ .

Normal equation

Set gradient to zero:  $\mathbf{w} = (X^\top X)^{-1} X^\top \mathbf{y}$  when inverse exists (columns of  $X$  linearly independent). Pseudoinverse handles singular/rank-deficient cases. Direct method: no learning rate/iterations, but matrix inversion costly with many features. Least squares still works when perfect prediction impossible.

**2x2 normal equation: every form**  
**Inverse**:  $\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \frac{1}{ad-bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$  (swap diag, negate off-diag,  $\div$  det).  $\det = 0 \Rightarrow$  dependent features, no inverse.  
(1)  $h = w_0 + w_1 x$  from data:  $X^\top X = \begin{pmatrix} n & \sum x \\ \sum x & \sum x^2 \end{pmatrix}$ ,  $X^\top \mathbf{y} = \begin{pmatrix} \sum y \\ \sum xy \end{pmatrix}$ .  
Check:  $w_1 = \frac{\sum (x-\bar{x})(y-\bar{y})}{\sum (x-\bar{x})^2}$ ,  $w_0 = \bar{y} - w_1 \bar{x}$  (line passes through  $(\bar{x}, \bar{y})$ ). *Ex:* (0, 0), (1, 1), (2, 3):  $\begin{pmatrix} 3 & 3 \\ 3 & 5 \end{pmatrix}$ ,  $\begin{pmatrix} 4 \\ 7 \end{pmatrix}$ ,  $\det 6$ ,  $\mathbf{w} = \frac{1}{6} \begin{pmatrix} 5 & -3 \\ -3 & 3 \end{pmatrix} \begin{pmatrix} 4 \\ 7 \end{pmatrix} = (-\frac{1}{6}, \frac{3}{2})$ .  
(2) **Two features, no intercept** ( $X$  has columns  $x_1, x_2$ ):  $X^\top X = \begin{pmatrix} \sum x_1^2 & \sum x_1 x_2 \\ \sum x_1 x_2 & \sum x_2^2 \end{pmatrix}$ ,  $X^\top \mathbf{y} = \begin{pmatrix} \sum x_1 y \\ \sum x_2 y \end{pmatrix}$ . Entry  $(j, k)$  = column  $j$ -column  $k$ .  
(3) **Diagonal**  $X^\top X$  (orthogonal columns):  $w_j = \frac{(X^\top \mathbf{y})_j}{(X^\top X)_{jj}}$ , i.e. invert each diagonal entry. *Ex:*  $X = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$ ,  $\mathbf{y} = \begin{pmatrix} 4 \\ 2 \end{pmatrix}$ :  $X^\top X = \text{diag}(2, 2)$ ,  $X^\top \mathbf{y} = (6, 2)$ , so  $\mathbf{w} = (3, 1)$ .  
(4)  **$X$  square & invertible** ( $n = \#$ weights):  $\mathbf{w} = X^{-1} \mathbf{y}$  directly, perfect fit,  $J = 0$ .  
(5) **1 feature, no intercept**:  $w = \sum xy / \sum x^2$ .  
**Shapes**:  $X$  is  $n \times (d+1)$ ;  $X^\top X$  is  $(d+1) \times (d+1)$ ;  $X^\top \mathbf{y}$  is  $(d+1) \times 1$ . Always solve  $X^\top X \mathbf{w} = X^\top \mathbf{y}$ ; sanity-check by plugging  $\mathbf{w}$  back in.

Gradient descent

$\mathbf{w} \leftarrow \mathbf{w} - \gamma \mathbf{g}$  ( $\gamma$  = learning rate,  $\mathbf{g}$  = gradient). Update all weights at once, from the old  $\mathbf{w}$ .  

- $\gamma$  too small: slow. Too big: bounces around or blows up.
- MSE for linear regression is a bowl (convex): GD reaches the global minimum.
- Feature scaling makes the bowl rounder, so GD converges faster.
- Batch**: all points per step (1 step per epoch). **SGD**: 1 point per step ( $n$  steps per epoch, noisy). **Mini-batch**:  $B$  points per step ( $n/B$  steps per epoch). Epoch = one pass over the data.

Feature transformations

Add new columns made from the old ones ( $x^2, x_1 x_2, \dots$ ), then do ordinary linear regression. The model is still linear in the weights, so MSE stays convex. Higher degree fits better but overfits more easily.

9. Linear regression: calculation recipes

**One GD step (plain / polynomial / any features)**  
1. Features of each point with a 1 in front:  $\mathbf{x} = (1, x)$ ; degree 2:  $\mathbf{x} = (1, x, x^2)$ .  
2. Prediction  $h = \mathbf{w} \cdot \mathbf{x}$ ; error  $r = h - y$ .  
3. Gradient  $\mathbf{g} = \frac{1}{n} \sum \mathbf{r} \mathbf{x}$  (scale each  $\mathbf{x}$  by its  $r$ , add, divide by  $n$ ).  
4.  $\mathbf{w} \leftarrow \mathbf{w} - \gamma \mathbf{g}$ . Loss  $J = \frac{1}{2n} \sum r^2$ .  
*Ex:*  $w = (4, -2, 1)$ , degree 2,  $\gamma = \frac{1}{2}$ , points  $(x, y) = (-1, 1), (1, -1), (2, 2)$ .  

|     |              |     |     |     |                         |
|-----|--------------|-----|-----|-----|-------------------------|
| $x$ | $\mathbf{x}$ | $h$ | $y$ | $r$ | $\mathbf{r} \mathbf{x}$ |
| -1  | (1, -1, 1)   | 7   | 1   | 6   | (6, -6, 6)              |
| 1   | (1, 1, 1)    | 3   | -1  | 4   | (4, 4, 4)               |
| 2   | (1, 2, 4)    | 4   | 2   | 2   | (2, 4, 8)               |
| sum |              |     |     |     | (12, 2, 18)             |

  
 $g = (12, 2, 18)/3 = (4, 0.67, 6)$ ;  $w = (4, -2, 1) - \frac{1}{2}(4, 0.67, 6) = (2, -2.33, -2)$ ;  
 $J = \frac{36+16+4}{6} = 9.33$ .

**Non-standard  $J$ : differentiate what is written**  
**Rule**:  $J = c \sum_i L(r_i)$ ,  $r = h - y$ .  $\mathbf{g}$  has one entry per weight, each computed separately:  $g_j = \partial J / \partial w_j = c \sum_i L'(r_i) x_j^{(i)}$  ( $x_0 = 1$  for  $w_0$ ). Table:  $\mathbf{x}$  = all  $g_j$  at once; for  $g_j$  read  $x_j$ .  

|                                   |                                          |                                  |                                              |
|-----------------------------------|------------------------------------------|----------------------------------|----------------------------------------------|
| $J$                               | $g$                                      | $J$                              | $g$                                          |
| $\frac{1}{2n} \sum r^2$           | $\frac{1}{n} \sum \mathbf{r} \mathbf{x}$ | $\frac{1}{2} \sum r^2$ (no $n$ ) | $\sum \mathbf{r} \mathbf{x}$                 |
| $\frac{1}{n} \sum r^2$            | $\frac{2}{n} \sum \mathbf{r} \mathbf{x}$ | $\sum r^2$                       | $2 \sum \mathbf{r} \mathbf{x}$               |
| $\frac{1}{2} \sum \alpha_i r_i^2$ | $\sum \alpha_i r_i \mathbf{x}$           | $\frac{1}{n} \sum  r $ (MAE)     | $\frac{1}{n} \sum \text{sign}(r) \mathbf{x}$ |
| $\frac{1}{n} \sum r^4$            | $\frac{4}{n} \sum r^3 \mathbf{x}$        | $\frac{1}{n} \sum f(r)$          | $\frac{1}{n} \sum f'(r) \mathbf{x}$          |

  
*Ex* (AY24/25 S2):  $J = \frac{1}{2} r_0^2 + \frac{1}{2} r_1^2$ ,  $w = (0.5, 1)$ , pts (0, 0), (2, 1),  $\gamma = \frac{1}{4}$ :  
 $r = (0.5, 1.5)$ .  
 $g_0 = \sum r_i \cdot 1 = 2$  ( $x_0 = 1$  even when  $x = 0$ );  $g_1 = \sum r_i x_i = 0.5 \cdot 0 + 1.5 \cdot 2 = 3$   
 $w_0 = 0.5 - \frac{1}{4} \cdot 2 = 0$ ,  $w_1 = 1 - \frac{1}{4} \cdot 3 = \frac{1}{4}$ .  
**Not squared loss**  $\Rightarrow$  no normal equation, use GD. Non-convex (e.g.  $\sqrt{|r|}$ )  $\Rightarrow$  SGD (noise escapes local minima).

**SGD**: one given point,  $\mathbf{g} = \mathbf{r} \mathbf{x}$  (no  $\div n$ ). **Mini-batch**: batch only,  $\div$  batch size.  
**Scaling**: min-max:  $\frac{x - \min}{\max - \min}$ . Standardise:  $\frac{x - \text{mean}}{\text{std}}$ .  
**Learning rate** (one weight,  $\mathbf{g} = a(w - w^*)$ ): converges if  $\gamma < \frac{2}{a}$ ; one step if  $\gamma = \frac{1}{a}$ ; zig-zags if  $\frac{1}{a} < \gamma < \frac{2}{a}$ .  
**DT “leaf  $\geq k$ ” pruning (sec. 7), e.g. hiring** (sec. 7), e.g. hiring: the  $>3y$  branch has 1 row  $\Rightarrow$  cut the whole Experience split  $\Rightarrow$  every candidate gets the root majority, **Yes** (5:2), even  $<1y$  ones.